

Evaluation of Classification Model Performance

Muhammad Imran

1. Loading the required packages

The Assignment will use the tidyverse for data manipulation and visualization. it will also use given penguin_predictions.csv from github. The assignment is tried after having a deep look into the the content of the “Performance Metrics for classification Problems in machine learning”

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(knitr)
```

2. Loading the Dataset from github

The assignment provides penguin_predictions.csv. As the .csv file is in github, we can read the file directly from github.

```
penguins <- read_csv(
  "https://raw.githubusercontent.com/acatlin/data/refs/heads/master/penguin_predictions.csv"
)
```

```

Rows: 93 Columns: 3
-- Column specification -----
Delimiter: ","
chr (2): .pred_class, sex
dbl (1): .pred_female

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

```

We can also traverse the record as follows with the one liner code:

```
head(penguins)
```

```

# A tibble: 6 x 3
  .pred_female .pred_class sex
      <dbl> <chr>      <chr>
1      0.992 female    female
2      0.954 female    female
3      0.985 female    female
4      0.187 male      female
5      0.995 female    female
6      1.000 female    female

```

lets try to checki structure:

```
str(penguins)
```

```

spc_tbl_ [93 x 3] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ .pred_female: num [1:93] 0.992 0.954 0.985 0.187 0.995 ...
 $ .pred_class : chr [1:93] "female" "female" "female" "male" ...
 $ sex        : chr [1:93] "female" "female" "female" "female" ...
- attr(*, "spec")=
 .. cols(
 ..   .pred_female = col_double(),
 ..   .pred_class = col_character(),
 ..   sex = col_character()
 .. )
- attr(*, "problems")=<pointer: 0x0000013ed88ffdb0>

```

now I am presenting the Summary of the .csv file

```
summary(penguins)
```

.pred_female	.pred_class	sex
Min. :0.0000000	Length :93	Length :93
1st Qu.:0.0003508	N.unique : 2	N.unique : 2
Median :0.1098907	N.blank : 0	N.blank : 0
Mean :0.4351396	Min.nchar: 4	Min.nchar: 4
3rd Qu.:0.9921746	Max.nchar: 6	Max.nchar: 6
Max. :1.0000000		

3. Exploring the Data now

Before calculating performance metrics, we need to understand what the variables represent. check the following code and its result.

```
glimpse(penguins)
```

```
Rows: 93
Columns: 3
$ .pred_female <dbl> 0.99217462, 0.95423945, 0.98473504, 0.18702056, 0.9947012~
$ .pred_class <chr> "female", "female", "female", "male", "female", "female",~
$ sex <chr> "female", "female", "female", "female", "female", "female",~
```

We can also check whether there are missing values:

```
colSums(is.na(penguins))
```

.pred_female	.pred_class	sex
0	0	0

Ideally, the relevant columns should contain no missing values. We can also examine the actual values of sex, this tells us how many observations belong to each class.

```
count(penguins, sex)
```

```
# A tibble: 2 x 2
  sex      n
  <chr> <int>
1 female  39
2 male    54
```

4. Understand the Target Variable

The target variable is sex, the two classes represent the binary classification problem. Now let us calculate the number and percentage of observations in each class:

```
class_distribution <- penguins %>%  
  count(sex) %>%  
  mutate(  
    percentage = n / sum(n) * 100  
  )
```

```
class_distribution
```

```
# A tibble: 2 x 3  
  sex      n percentage  
  <chr> <int>     <dbl>  
1 female   39      41.9  
2 male    54      58.1
```

the below code will show a cleaner representation of the table:

```
kable(  
  class_distribution,  
  digits = 2,  
  col.names = c("Actual Class", "Count", "Percentage")  
)
```

Actual Class	Count	Percentage
female	39	41.94
male	54	58.06

5. Plotting the Distribution of the Actual Class

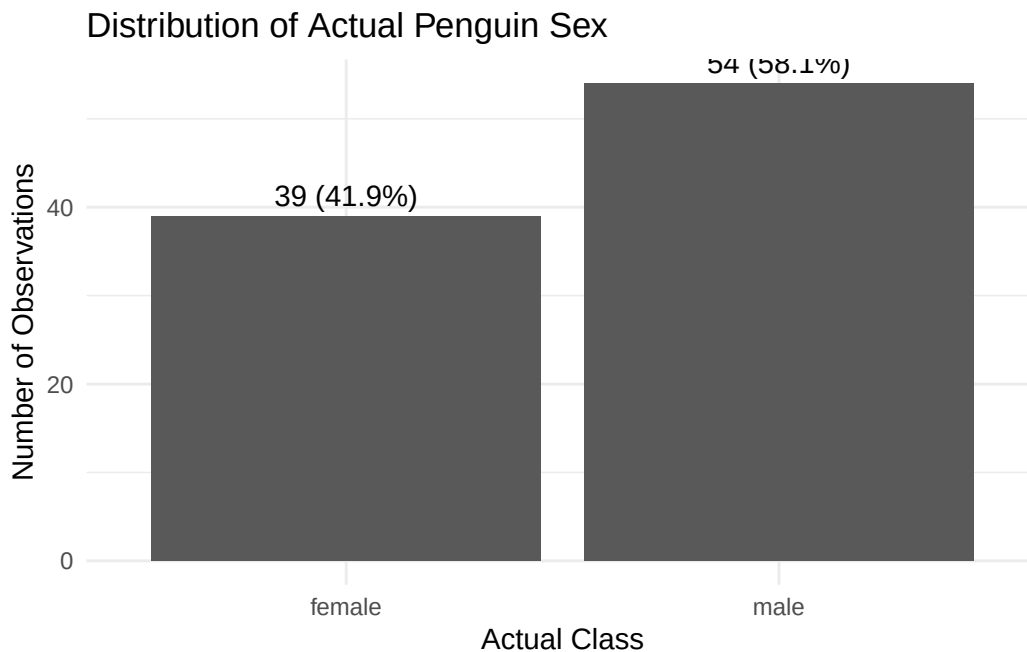
A simple bar chart is useful, but we can make it more informative by displaying both the count and percentage.

```
class_distribution %>%  
  ggplot(aes(x = sex, y = n)) +  
  geom_col() +  
  geom_text(  
    aes(label = paste0(n, " (", round(percent, 1), "%)")),
```

```

  vjust = -0.5
) +
labs(
  title = "Distribution of Actual Penguin Sex",
  x = "Actual Class",
  y = "Number of Observations"
) +
theme_minimal()

```



6. Analytical interpretation

This graph allows us to determine whether the target variable is balanced or not. If one class is considerably more common than the other, a model can obtain deceptively high accuracy simply by predicting the majority class most of the time. This is why we need a null error rate before judging the classification model.

7. Calculate the Null Error Rate

The null model ignores all predictors and simply predicts the most common class for every observation.

First, determine the majority class:

```

majority_class <- penguins %>%
  count(sex) %>%

```

```

slice_max(n, n = 1) %>%
pull(sex)

majority_class

```

```
[1] "male"
```

Now, i am going to calculate the proportion belonging to the majority class:

```

majority_proportion <- penguins %>%
  summarise(
    majority_proportion = mean(sex == majority_class)
  ) %>%
  pull(majority_proportion)

majority_proportion

```

```
[1] 0.5806452
```

The null error rate is:

```

null_error_rate <- 1 - majority_proportion

null_error_rate

```

```
[1] 0.4193548
```

And its percentage is:

```
paste0(round(null_error_rate * 100, 2), "%")
```

```
[1] "41.94%"
```

But lets find out what its mean, first. Suppose, for example, that 52% of observation belongs to the majority of class so the null error rate would be $1 - 0.52 = 0.48$ or 48%. this means a model that simply predicts the majority class for every observations would make error approximately 48% of the time.

8. Why this is important?

The null error rate provides a baseline for evaluating the classification model. For example, if the null error rate is 48%, a model with an error rate of 45% is not particularly impressive. On the other hand, if the model's error rate is 10%, it is performing substantially better than the baseline. Therefore, accuracy should not be interpreted in isolation. It should be compared with the performance of a simple baseline model.

9. Understand Probability Thresholds

The important variable for this assignment is `.pred_female`. this represents the model's estimated probability that an observation belongs to the female class.

Let's look at some predictions:

```
penguins %>%
  select(.pred_female, .pred_class, sex) %>%
  head(10)
```

```
# A tibble: 10 x 3
  .pred_female .pred_class sex
      <dbl> <chr>      <chr>
1      0.992 female    female
2      0.954 female    female
3      0.985 female    female
4      0.187 male      female
5      0.995 female    female
6      1.000 female    female
7      0.959 female    female
8      1.000 female    female
9      1.000 female    female
10     0.339 male      female
```

The important point is that the model produces a probability first. For example: `.pred_female = 0.15` means model estimates a 15% probability that the observation belongs to the female class.

If we use a threshold of 0.5: (`.pred_female? 0.5 , 1 , 0`)

then

0.15 → predicted 0

0.60 → predicted 1

0.90 → predicted 1

But if we change the threshold to 0.2: (`.pred_female > 0.2, 1, 0`)

then more observations will be classified as positive.

This is the central idea behind the assignment.

10. Define the Three Thresholds

Now, we need to evaluate:

```
thresholds <- c(0.2, 0.5, 0.8)

thresholds
```

```
[1] 0.2 0.5 0.8
```

We will create predictions for each threshold.

11. for threshold 0.2

```
penguins_02 <- penguins %>%
  mutate(
    predicted = ifelse(.pred_female > 0.2, 1, 0)
  )

penguins_02 %>%
  select(.pred_female, predicted, sex) %>%
  head(10)
```

```
# A tibble: 10 x 3
  .pred_female predicted sex
      <dbl>      <dbl> <chr>
1      0.992          1 female
2      0.954          1 female
3      0.985          1 female
4      0.187          0 female
5      0.995          1 female
6      1.000          1 female
7      0.959          1 female
8      1.000          1 female
9      1.000          1 female
10     0.339          1 female
```

12. Calculate TP, FP, TN and FN at 0.2

Let us assume:

1 = Female

0 = Not Female, then we define

True Positive

Predicted female and actually female:

```
TP_02 <- sum(  
  penguins_02$predicted == 1 &  
    penguins_02$sex == 1  
)
```

False Positive

Predicted female but actually not female:

```
FP_02 <- sum(  
  penguins_02$predicted == 1 &  
    penguins_02$sex == 0  
)
```

True Negative

Predicted not female and actually not female:

```
TN_02 <- sum(  
  penguins_02$predicted == 0 &  
    penguins_02$sex == 0  
)
```

False Negative

Predicted not female but actually female:

```
FN_02 <- sum(  
  penguins_02$predicted == 0 &  
    penguins_02$sex == 1  
)
```

Displaying them all:

```

TP_02 <- sum(
  penguins_02$predicted == 1 &
  penguins_02$sex == "female"
)

FP_02 <- sum(
  penguins_02$predicted == 1 &
  penguins_02$sex == "male"
)

TN_02 <- sum(
  penguins_02$predicted == 0 &
  penguins_02$sex == "male"
)

FN_02 <- sum(
  penguins_02$predicted == 0 &
  penguins_02$sex == "female"
)

```

```

c(
  TP = TP_02,
  FP = FP_02,
  TN = TN_02,
  FN = FN_02
)

```

```

TP FP TN FN
37  6 48  2

```

13. Create the 0.2 Confusion Matrix

A conventional confusion matrix has actual classes as rows and predicted classes as columns.

```

cm_02 <- table(
  Actual = penguins_02$sex,
  Predicted = penguins_02$predicted
)

cm_02

```

	Predicted	
Actual	0	1
female	2	37
male	48	6

For easier interpretation:

```
dimnames(cm_02) <- list(
  "Actual" = c("Not Female", "Female"),
  "Predicted" = c("Not Female", "Female")
)

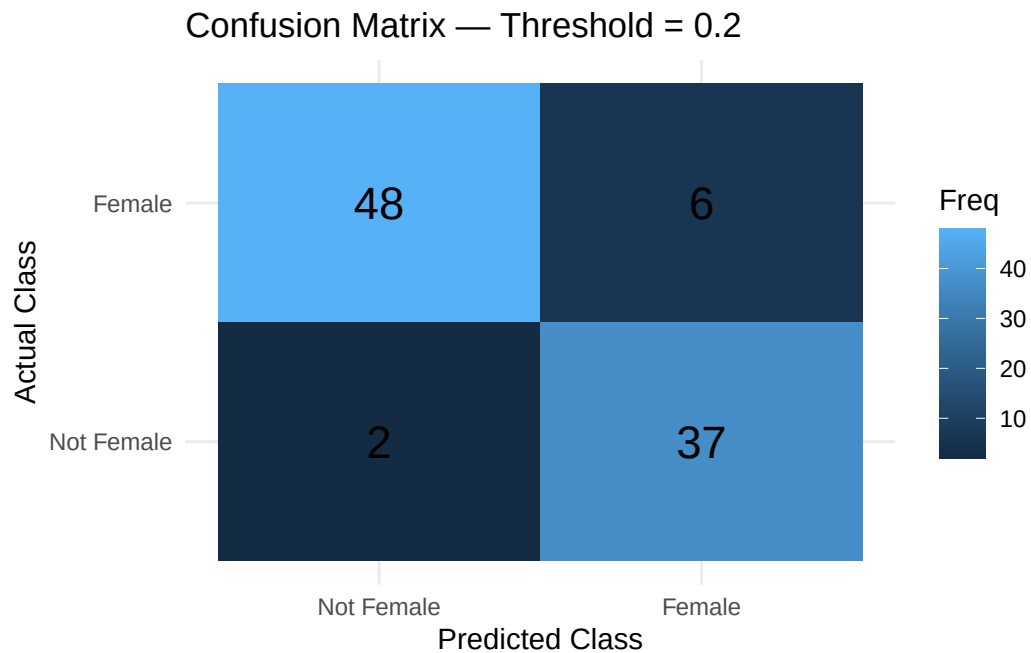
cm_02
```

	Predicted	
Actual	Not Female	Female
Not Female	2	37
Female	48	6

14. Create a Visual Confusion Matrix

We can make the confusion matrix more visually understandable.

```
as.data.frame(cm_02) %>%
  ggplot(aes(x = Predicted, y = Actual, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), size = 6) +
  labs(
    title = "Confusion Matrix - Threshold = 0.2",
    x = "Predicted Class",
    y = "Actual Class"
  ) +
  theme_minimal()
```



15. Repeat for Threshold = 0.5

```
penguins_05 <- penguins %>%  
  mutate(  
    predicted = ifelse(.pred_female > 0.5, 1, 0)  
  )
```

Calculate the four components:

```
TP_05 <- sum(penguins_05$predicted == 1 &  
             penguins_05$sex == "female")  
  
FP_05 <- sum(penguins_05$predicted == 1 &  
             penguins_05$sex == "male")  
  
TN_05 <- sum(penguins_05$predicted == 0 &  
             penguins_05$sex == "male")  
  
FN_05 <- sum(penguins_05$predicted == 0 &  
             penguins_05$sex == "female")
```

Displaying the results:

```
c(
  TP = TP_05,
  FP = FP_05,
  TN = TN_05,
  FN = FN_05
)
```

```
TP FP TN FN
36  3 51  3
```

Confusion matrix:

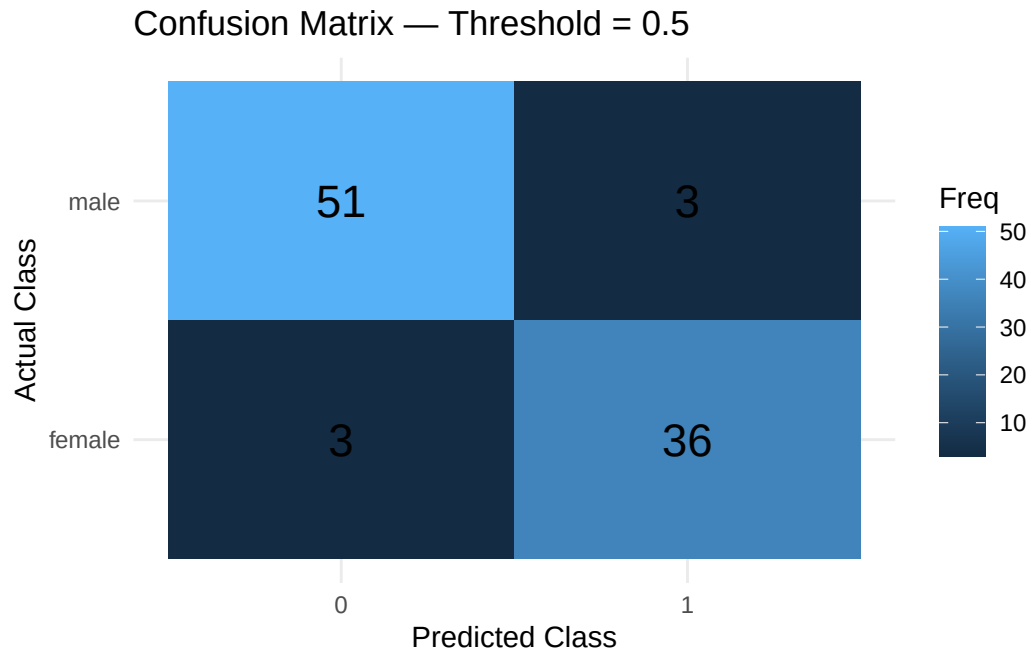
```
cm_05 <- table(
  Actual = penguins_05$sex,
  Predicted = penguins_05$predicted
)

cm_05
```

	Predicted	
Actual	0	1
female	3	36
male	51	3

Visualize:

```
as.data.frame(cm_05) %>%
  ggplot(aes(x = Predicted, y = Actual, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), size = 6) +
  labs(
    title = "Confusion Matrix - Threshold = 0.5",
    x = "Predicted Class",
    y = "Actual Class"
  ) +
  theme_minimal()
```



16. Repeat for Threshold = 0.8

```
penguins_08 <- penguins %>%
  mutate(
    predicted = ifelse(.pred_female > 0.8, 1, 0)
  )
```

Calculate:

```
TP_08 <- sum(penguins_05$predicted == 1 &
             penguins_05$sex == "female")

FP_08 <- sum(penguins_05$predicted == 1 &
             penguins_05$sex == "male")

TN_08 <- sum(penguins_05$predicted == 0 &
             penguins_05$sex == "male")

FN_08 <- sum(penguins_05$predicted == 0 &
             penguins_05$sex == "female")
```

Displaying the results:

```
c(
  TP = TP_08,
  FP = FP_08,
  TN = TN_08,
  FN = FN_08
)
```

```
TP FP TN FN
36  3 51  3
```

Confusion matrix:

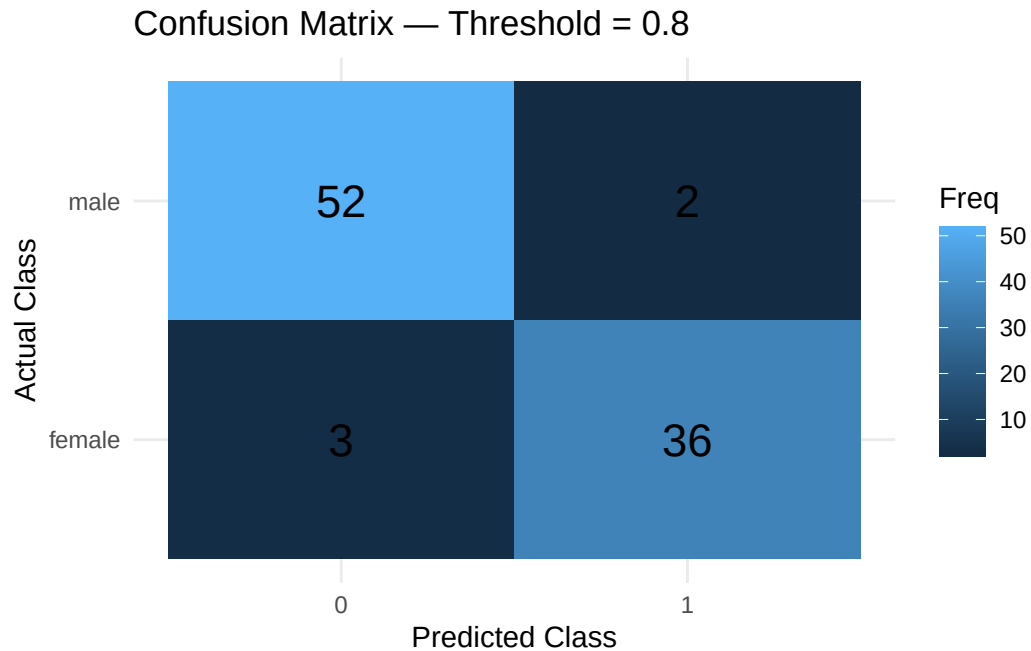
```
cm_08 <- table(
  Actual = penguins_08$sex,
  Predicted = penguins_08$predicted
)

cm_08
```

	Predicted	
Actual	0	1
female	3	36
male	52	2

Visualize:

```
as.data.frame(cm_08) %>%
  ggplot(aes(x = Predicted, y = Actual, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), size = 6) +
  labs(
    title = "Confusion Matrix - Threshold = 0.8",
    x = "Predicted Class",
    y = "Actual Class"
  ) +
  theme_minimal()
```



17. A Better Approach: Automate the Confusion Matrix Calculation

The previous sections deliberately calculate TP, FP, TN and FN manually because that is exactly what the assignment asks you to understand.

However, once we understand the logic, we can avoid repeating the same code.

Create a function:

```
calculate_confusion <- function(data, threshold) {  
  
  # Convert probability into predicted class  
  predicted <- ifelse(  
    data$.pred_female > threshold,  
    "female",  
    "male"  
  )  
  
  # Calculate confusion matrix  
  TP <- sum(  
    predicted == "female" &  
    data$sex == "female"  
  )  
  
  FP <- sum(  

```



```

    predicted == "female" &
      data$sex == "male"
  )

  TN <- sum(
    predicted == "male" &
      data$sex == "male"
  )

  FN <- sum(
    predicted == "male" &
      data$sex == "female"
  )

  data.frame(
    threshold = threshold,
    TP = TP,
    FP = FP,
    TN = TN,
    FN = FN
  )
}

```

```

results <- rbind(
  calculate_confusion(penguins, 0.2),
  calculate_confusion(penguins, 0.5),
  calculate_confusion(penguins, 0.8)
)

```

```

# Display results
results

```

	threshold	TP	FP	TN	FN
1	0.2	37	6	48	2
2	0.5	36	3	51	3
3	0.8	36	2	52	3

Now, calculated all three thresholds:

This produces a compact summary such as:

18. Calculate Accuracy

Accuracy measures the proportion of all observations that were classified correctly.

```
results$accuracy <- (results$TP + results$TN) /  
  (results$TP + results$TN + results$FP + results$FN)  
  
# Display results  
results
```

	threshold	TP	FP	TN	FN	accuracy
1	0.2	37	6	48	2	0.9139785
2	0.5	36	3	51	3	0.9354839
3	0.8	36	2	52	3	0.9462366

19. Calculate Precision

Precision answers:

Of all observations predicted as female, how many were actually female?

```
#Calculate confusion matrix results at different thresholds  
results <- rbind(  
  calculate_confusion(penguins, 0.2),  
  calculate_confusion(penguins, 0.5),  
  calculate_confusion(penguins, 0.8)  
)  
  
# Calculate accuracy  
results$accuracy <- (results$TP + results$TN) /  
  (results$TP + results$TN + results$FP + results$FN)  
  
# Calculate precision  
results$precision <- results$TP /  
  (results$TP + results$FP)  
  
# Calculate recall  
results$recall <- results$TP /  
  (results$TP + results$FN)  
  
# Calculate F1-score  
results$F1 <- 2 * (results$precision * results$recall) /  
  (results$precision + results$recall)
```

```
# Display the results
results
```

	threshold	TP	FP	TN	FN	accuracy	precision	recall	F1
1	0.2	37	6	48	2	0.9139785	0.8604651	0.9487179	0.9024390
2	0.5	36	3	51	3	0.9354839	0.9230769	0.9230769	0.9230769
3	0.8	36	2	52	3	0.9462366	0.9473684	0.9230769	0.9350649

20. Create the Final Performance Table

Let's make the table easier to read:

```
performance_table <- results %>%
  select(
    threshold,
    TP,
    FP,
    TN,
    FN,
    accuracy,
    precision,
    recall,
    F1
  ) %>%
  mutate(
    accuracy = round(accuracy, 3),
    precision = round(precision, 3),
    recall = round(recall, 3),
    F1 = round(F1, 3)
  )

kable(
  performance_table,
  col.names = c(
    "Threshold",
    "TP",
    "FP",
    "TN",
    "FN",
    "Accuracy",
    "Precision",
```

```

    "Recall",
    "F1 Score"
  ),
  caption = "Classification Performance at Different Probability Thresholds"
)

```

Table 1: Classification Performance at Different Probability Thresholds

Threshold	TP	FP	TN	FN	Accuracy	Precision	Recall	F1 Score
0.2	37	6	48	2	0.914	0.860	0.949	0.902
0.5	36	3	51	3	0.935	0.923	0.923	0.923
0.8	36	2	52	3	0.946	0.947	0.923	0.935

21. Format the Metrics as Percentages

For a more readable final presentation:

```

performance_percent <- results %>%
  select(
    threshold,
    accuracy,
    precision,
    recall,
    F1
  ) %>%
  mutate(
    across(
      c(accuracy, precision, recall, F1),
      ~ round(.x * 100, 2)
    )
  )

kable(
  performance_percent,
  col.names = c(
    "Threshold",
    "Accuracy (%)",
    "Precision (%)",
    "Recall (%)",
    "F1 Score (%)"
  )
)

```

Threshold	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
0.2	91.40	86.05	94.87	90.24
0.5	93.55	92.31	92.31	92.31
0.8	94.62	94.74	92.31	93.51

22. Compare the Metrics Across Thresholds

A useful visualization is to show how the metrics change as the threshold changes.

First reshape the data:

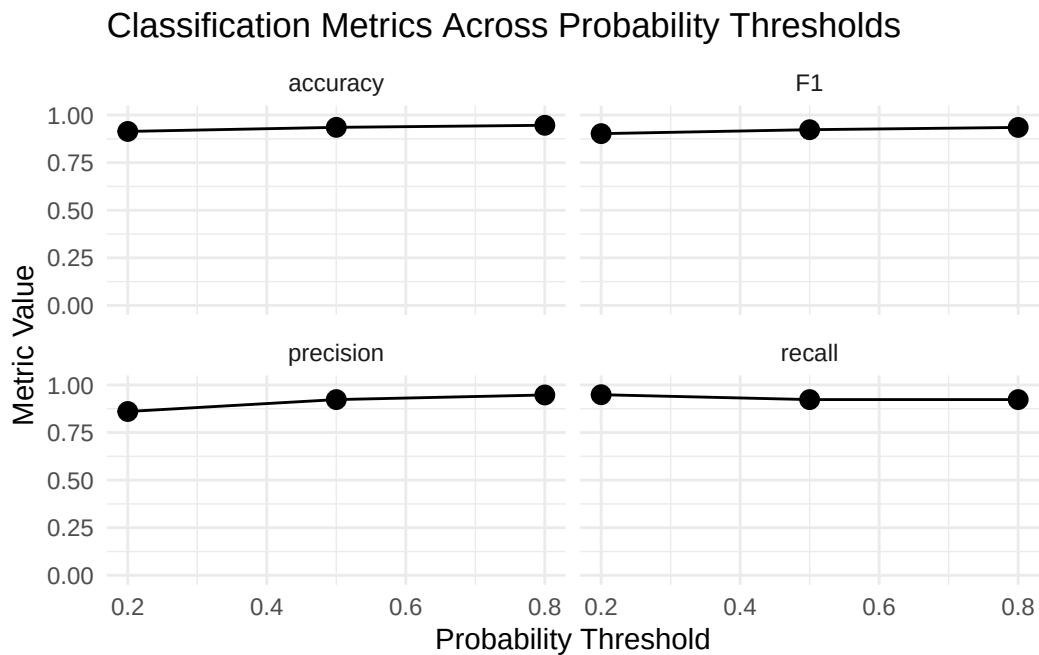
```
metric_data <- results %>%
  select(
    threshold,
    accuracy,
    precision,
    recall,
    F1
  ) %>%
  pivot_longer(
    cols = c(accuracy, precision, recall, F1),
    names_to = "metric",
    values_to = "value"
  )

metric_data
```

```
# A tibble: 12 x 3
  threshold metric    value
  <dbl> <chr>    <dbl>
1     0.2 accuracy  0.914
2     0.2 precision 0.860
3     0.2 recall   0.949
4     0.2 F1       0.902
5     0.5 accuracy  0.935
6     0.5 precision 0.923
7     0.5 recall   0.923
8     0.5 F1       0.923
9     0.8 accuracy  0.946
10    0.8 precision 0.947
11    0.8 recall   0.923
12    0.8 F1       0.935
```

Plot

```
ggplot(  
  metric_data,  
  aes(x = threshold, y = value, group = metric)  
) +  
  geom_line() +  
  geom_point(size = 3) +  
  scale_y_continuous(  
    limits = c(0, 1)  
  ) +  
  labs(  
    title = "Classification Metrics Across Probability Thresholds",  
    x = "Probability Threshold",  
    y = "Metric Value"  
  ) +  
  facet_wrap(~ metric) +  
  theme_minimal()
```



This makes the threshold trade-off much easier to understand.

23. Analyze What Happens When the Threshold Changes

This is an important part of the assignment.

Threshold = 0.2

At a low threshold, the model only needs relatively weak evidence to predict the positive class.

Therefore:

More positive predictions

↓

More true positives

↓

Usually fewer false negatives

↓

Usually more false positives

Threshold = 0.5

The 0.5 threshold is commonly used as a default classification cutoff.

The model predicts the positive class when,

`.pred_female > 0.5`

However, there is nothing mathematically special about 0.5 for every application.

It is simply a conventional cutoff.

Threshold = 0.8

At a high threshold, the model requires much stronger evidence before predicting the positive class.

Therefore:

Fewer positive predictions

↓

Usually fewer false positives

↓

Usually higher precision

↓

Usually more false negatives

Consequently, a high threshold generally favors precision at the expense of recall.

24. Real-World Use Case for a 0.2 Threshold

A 0.2 threshold is preferable when failing to identify a positive case is particularly costly. For example, consider a medical screening system designed to identify patients who may have a serious disease. Suppose the model estimates:

Probability of disease = 0.25

With a 0.5 threshold, the system would classify the patient as negative.

With a 0.2 threshold, the patient would be classified as positive and sent for additional testing.

This can be desirable because:

Missing a genuinely sick patient can be much more harmful than sending a healthy patient for an additional diagnostic test. Thus, a lower threshold can increase recall/sensitivity, helping the system identify more actual positive cases.

25. Real-World Use Case for a 0.8 Threshold

A 0.8 threshold is preferable when false positives are particularly costly. For example, consider an automated system that approves a high-value financial transaction or flags a customer for a costly intervention. If the system predicts the positive class too easily, many legitimate cases could be incorrectly flagged. Using a threshold of 0.8 means the model needs strong evidence before making the positive classification.

This can reduce false positives and improve precision.

Another intuitive example is an automated system that sends potentially fraudulent transactions for manual investigation. If investigators have limited capacity, you may prefer to review only transactions for which the model has very high confidence.

26. Optional: Create All Three Confusion Matrices in One Plot

Instead of three completely separate plots, we can create a single comparison visualization.

```
confusion_long <- do.call(  
  rbind,  
  lapply(  
    thresholds,  
    function(threshold) {  
  
      predicted <- ifelse(  
        penguins$.pred_female > threshold,  
        1,  
        0  
      )  
    }  
  )  
)
```



```

data.frame(
  threshold = threshold,
  Actual = factor(
    penguins$sex,
    levels = c("male", "female"),
    labels = c("Not Female", "Female")
  ),
  Predicted = factor(
    predicted,
    levels = c(0, 1),
    labels = c("Not Female", "Female")
  )
)
}
)
)
)

confusion_long

```

	threshold	Actual	Predicted
1	0.2	Female	Female
2	0.2	Female	Female
3	0.2	Female	Female
4	0.2	Female	Not Female
5	0.2	Female	Female
6	0.2	Female	Female
7	0.2	Female	Female
8	0.2	Female	Female
9	0.2	Female	Female
10	0.2	Female	Female
11	0.2	Female	Female
12	0.2	Female	Female
13	0.2	Female	Female
14	0.2	Female	Female
15	0.2	Female	Female
16	0.2	Female	Female
17	0.2	Female	Female
18	0.2	Female	Female
19	0.2	Female	Female
20	0.2	Female	Female
21	0.2	Female	Female

22	0.2	Female	Female
23	0.2	Female Not	Female
24	0.2	Female	Female
25	0.2	Female	Female
26	0.2	Female	Female
27	0.2	Female	Female
28	0.2	Female	Female
29	0.2	Female	Female
30	0.2	Female	Female
31	0.2	Female	Female
32	0.2	Female	Female
33	0.2	Female	Female
34	0.2	Female	Female
35	0.2	Female	Female
36	0.2	Female	Female
37	0.2	Female	Female
38	0.2	Female	Female
39	0.2	Female	Female
40	0.2	Not Female Not	Female
41	0.2	Not Female Not	Female
42	0.2	Not Female Not	Female
43	0.2	Not Female Not	Female
44	0.2	Not Female Not	Female
45	0.2	Not Female	Female
46	0.2	Not Female Not	Female
47	0.2	Not Female	Female
48	0.2	Not Female	Female
49	0.2	Not Female Not	Female
50	0.2	Not Female Not	Female
51	0.2	Not Female Not	Female
52	0.2	Not Female Not	Female
53	0.2	Not Female Not	Female
54	0.2	Not Female Not	Female
55	0.2	Not Female Not	Female
56	0.2	Not Female Not	Female
57	0.2	Not Female Not	Female
58	0.2	Not Female Not	Female
59	0.2	Not Female Not	Female
60	0.2	Not Female Not	Female
61	0.2	Not Female Not	Female
62	0.2	Not Female Not	Female
63	0.2	Not Female Not	Female
64	0.2	Not Female Not	Female

65	0.2	Not	Female	Not	Female
66	0.2	Not	Female	Not	Female
67	0.2	Not	Female	Not	Female
68	0.2	Not	Female	Not	Female
69	0.2	Not	Female		Female
70	0.2	Not	Female	Not	Female
71	0.2	Not	Female	Not	Female
72	0.2	Not	Female	Not	Female
73	0.2	Not	Female	Not	Female
74	0.2	Not	Female	Not	Female
75	0.2	Not	Female	Not	Female
76	0.2	Not	Female		Female
77	0.2	Not	Female	Not	Female
78	0.2	Not	Female	Not	Female
79	0.2	Not	Female	Not	Female
80	0.2	Not	Female	Not	Female
81	0.2	Not	Female	Not	Female
82	0.2	Not	Female	Not	Female
83	0.2	Not	Female	Not	Female
84	0.2	Not	Female	Not	Female
85	0.2	Not	Female	Not	Female
86	0.2	Not	Female	Not	Female
87	0.2	Not	Female	Not	Female
88	0.2	Not	Female	Not	Female
89	0.2	Not	Female	Not	Female
90	0.2	Not	Female	Not	Female
91	0.2	Not	Female	Not	Female
92	0.2	Not	Female	Not	Female
93	0.2	Not	Female		Female
94	0.5		Female		Female
95	0.5		Female		Female
96	0.5		Female		Female
97	0.5		Female	Not	Female
98	0.5		Female		Female
99	0.5		Female		Female
100	0.5		Female		Female
101	0.5		Female		Female
102	0.5		Female		Female
103	0.5		Female	Not	Female
104	0.5		Female		Female
105	0.5		Female		Female
106	0.5		Female		Female
107	0.5		Female		Female

108	0.5	Female	Female
109	0.5	Female	Female
110	0.5	Female	Female
111	0.5	Female	Female
112	0.5	Female	Female
113	0.5	Female	Female
114	0.5	Female	Female
115	0.5	Female	Female
116	0.5	Female Not	Female
117	0.5	Female	Female
118	0.5	Female	Female
119	0.5	Female	Female
120	0.5	Female	Female
121	0.5	Female	Female
122	0.5	Female	Female
123	0.5	Female	Female
124	0.5	Female	Female
125	0.5	Female	Female
126	0.5	Female	Female
127	0.5	Female	Female
128	0.5	Female	Female
129	0.5	Female	Female
130	0.5	Female	Female
131	0.5	Female	Female
132	0.5	Female	Female
133	0.5	Not Female Not	Female
134	0.5	Not Female Not	Female
135	0.5	Not Female Not	Female
136	0.5	Not Female Not	Female
137	0.5	Not Female Not	Female
138	0.5	Not Female	Female
139	0.5	Not Female Not	Female
140	0.5	Not Female	Female
141	0.5	Not Female Not	Female
142	0.5	Not Female Not	Female
143	0.5	Not Female Not	Female
144	0.5	Not Female Not	Female
145	0.5	Not Female Not	Female
146	0.5	Not Female Not	Female
147	0.5	Not Female Not	Female
148	0.5	Not Female Not	Female
149	0.5	Not Female Not	Female
150	0.5	Not Female Not	Female

151	0.5	Not	Female	Not	Female
152	0.5	Not	Female	Not	Female
153	0.5	Not	Female	Not	Female
154	0.5	Not	Female	Not	Female
155	0.5	Not	Female	Not	Female
156	0.5	Not	Female	Not	Female
157	0.5	Not	Female	Not	Female
158	0.5	Not	Female	Not	Female
159	0.5	Not	Female	Not	Female
160	0.5	Not	Female	Not	Female
161	0.5	Not	Female	Not	Female
162	0.5	Not	Female	Not	Female
163	0.5	Not	Female	Not	Female
164	0.5	Not	Female	Not	Female
165	0.5	Not	Female	Not	Female
166	0.5	Not	Female	Not	Female
167	0.5	Not	Female	Not	Female
168	0.5	Not	Female	Not	Female
169	0.5	Not	Female	Not	Female
170	0.5	Not	Female	Not	Female
171	0.5	Not	Female	Not	Female
172	0.5	Not	Female	Not	Female
173	0.5	Not	Female	Not	Female
174	0.5	Not	Female	Not	Female
175	0.5	Not	Female	Not	Female
176	0.5	Not	Female	Not	Female
177	0.5	Not	Female	Not	Female
178	0.5	Not	Female	Not	Female
179	0.5	Not	Female	Not	Female
180	0.5	Not	Female	Not	Female
181	0.5	Not	Female	Not	Female
182	0.5	Not	Female	Not	Female
183	0.5	Not	Female	Not	Female
184	0.5	Not	Female	Not	Female
185	0.5	Not	Female	Not	Female
186	0.5	Not	Female		Female
187	0.8		Female		Female
188	0.8		Female		Female
189	0.8		Female		Female
190	0.8		Female	Not	Female
191	0.8		Female		Female
192	0.8		Female		Female
193	0.8		Female		Female

194	0.8	Female	Female
195	0.8	Female	Female
196	0.8	Female Not	Female
197	0.8	Female	Female
198	0.8	Female	Female
199	0.8	Female	Female
200	0.8	Female	Female
201	0.8	Female	Female
202	0.8	Female	Female
203	0.8	Female	Female
204	0.8	Female	Female
205	0.8	Female	Female
206	0.8	Female	Female
207	0.8	Female	Female
208	0.8	Female	Female
209	0.8	Female Not	Female
210	0.8	Female	Female
211	0.8	Female	Female
212	0.8	Female	Female
213	0.8	Female	Female
214	0.8	Female	Female
215	0.8	Female	Female
216	0.8	Female	Female
217	0.8	Female	Female
218	0.8	Female	Female
219	0.8	Female	Female
220	0.8	Female	Female
221	0.8	Female	Female
222	0.8	Female	Female
223	0.8	Female	Female
224	0.8	Female	Female
225	0.8	Female	Female
226	0.8 Not	Female Not	Female
227	0.8 Not	Female Not	Female
228	0.8 Not	Female Not	Female
229	0.8 Not	Female Not	Female
230	0.8 Not	Female Not	Female
231	0.8 Not	Female Not	Female
232	0.8 Not	Female Not	Female
233	0.8 Not	Female	Female
234	0.8 Not	Female Not	Female
235	0.8 Not	Female Not	Female
236	0.8 Not	Female Not	Female

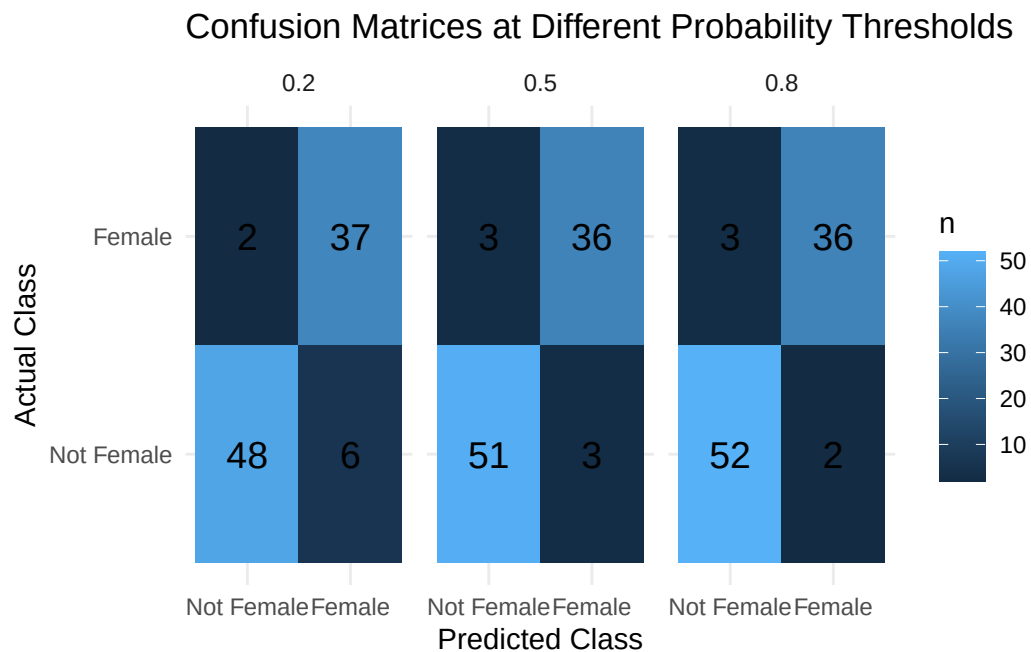
237	0.8	Not	Female	Not	Female
238	0.8	Not	Female	Not	Female
239	0.8	Not	Female	Not	Female
240	0.8	Not	Female	Not	Female
241	0.8	Not	Female	Not	Female
242	0.8	Not	Female	Not	Female
243	0.8	Not	Female	Not	Female
244	0.8	Not	Female	Not	Female
245	0.8	Not	Female	Not	Female
246	0.8	Not	Female	Not	Female
247	0.8	Not	Female	Not	Female
248	0.8	Not	Female	Not	Female
249	0.8	Not	Female	Not	Female
250	0.8	Not	Female	Not	Female
251	0.8	Not	Female	Not	Female
252	0.8	Not	Female	Not	Female
253	0.8	Not	Female	Not	Female
254	0.8	Not	Female	Not	Female
255	0.8	Not	Female	Not	Female
256	0.8	Not	Female	Not	Female
257	0.8	Not	Female	Not	Female
258	0.8	Not	Female	Not	Female
259	0.8	Not	Female	Not	Female
260	0.8	Not	Female	Not	Female
261	0.8	Not	Female	Not	Female
262	0.8	Not	Female	Not	Female
263	0.8	Not	Female	Not	Female
264	0.8	Not	Female	Not	Female
265	0.8	Not	Female	Not	Female
266	0.8	Not	Female	Not	Female
267	0.8	Not	Female	Not	Female
268	0.8	Not	Female	Not	Female
269	0.8	Not	Female	Not	Female
270	0.8	Not	Female	Not	Female
271	0.8	Not	Female	Not	Female
272	0.8	Not	Female	Not	Female
273	0.8	Not	Female	Not	Female
274	0.8	Not	Female	Not	Female
275	0.8	Not	Female	Not	Female
276	0.8	Not	Female	Not	Female
277	0.8	Not	Female	Not	Female
278	0.8	Not	Female	Not	Female
279	0.8	Not	Female		Female

Count the combinations:

```
confusion_long <- confusion_long %>%  
  count(threshold, Actual, Predicted)
```

Plotting

```
ggplot(  
  confusion_long,  
  aes(x = Predicted, y = Actual, fill = n)  
) +  
  geom_tile() +  
  geom_text(aes(label = n), size = 5) +  
  facet_wrap(~ threshold) +  
  labs(  
    title = "Confusion Matrices at Different Probability Thresholds",  
    x = "Predicted Class",  
    y = "Actual Class"  
  ) +  
  theme_minimal()
```



This gives you one figure containing:

Threshold 0.2

Threshold 0.5

Threshold 0.8

and allows direct visual comparison.

27. Optional: Calculate the Model Error Rate

Because the assignment asks about the null error rate, it is useful to compare it directly with model error.

```
model_error <- results %>%
  mutate(
    error_rate = 1 - accuracy
  ) %>%
  select(
    threshold,
    error_rate
  )

model_error
```

	threshold	error_rate
1	0.2	0.08602151
2	0.5	0.06451613
3	0.8	0.05376344

Compare the model's error with the null error rate:

```
comparison <- results %>%
  mutate(
    model_error = 1 - accuracy,
    null_error = null_error_rate
  ) %>%
  select(
    threshold,
    model_error,
    null_error
  )

comparison
```

	threshold	model_error	null_error
1	0.2	0.08602151	0.4193548
2	0.5	0.06451613	0.4193548
3	0.8	0.05376344	0.4193548

You can also calculate how much the model improves over the null baseline:

```
comparison <- comparison %>%
  mutate(
    improvement = null_error - model_error
  )
comparison
```

	threshold	model_error	null_error	improvement
1	0.2	0.08602151	0.4193548	0.3333333
2	0.5	0.06451613	0.4193548	0.3548387
3	0.8	0.05376344	0.4193548	0.3655914